

JAVA PROGRAMMING

UNIT 2 — COMPLETE NOTES

B.Tech 2nd Year | Exam Preparation

Syllabus

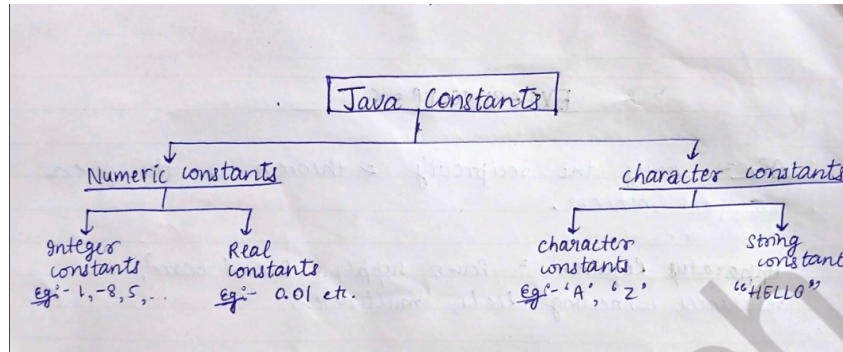
- Java Fundamentals, Data Types & Literals, Variables
- Wrapper Classes, Arrays
- Arithmetic, Relational, Logical, Bitwise & Other Operators
- Control of Flow — if, switch, while, do-while, for
- Classes, Objects, Constructors, Method Overloading
- Class Member Modifiers — static, final, abstract
- Inheritance, Method Overriding, Dynamic Method Dispatch
- Interfaces & Abstract Classes
- Anonymous Inner Classes
- Exception Handling — try, catch, finally, throw, throws
- User-Defined Exceptions
- String & StringBuffer Class
- StringTokenizer
- Applets — Life Cycle & Security Concerns

Best of Luck for IPU End-Sem Exams!

SECTION 1 — Java Fundamentals: Data Types, Variables & Constants

1.1 Constants in Java

Constants in Java refer to fixed values that do not change during the execution of a program. Java has two main types of constants:



Type	Sub-Type	Examples
Numeric Constants	Integer Constants	1, 2, 3, -10, 255
Numeric Constants	Real Constants	0.01, 3.14, -2.5
Character Constants	Character Constants	'x', 'y', 'A'
Character Constants	String Constants	"Hello", "Java"

1.2 Variables

A variable is an identifier that denotes a storage location used to store a data value. Variables can change value during program execution.

Declaration of Variables:

```

type variableName;
int count;
float x, y;
double pi;
  
```

Assignment Statement:

```

variableName = value;
yes = 'x';
z = 0;
  
```

Scope of Variables — 3 Types:

Variable Type	Where Declared	Scope / Lifetime	Key Point
Instance Variables	Inside class, outside methods	Lifetime = object lifetime	Different value for each object
Class Variables (Static Variable)	Inside class with static keyword	Lifetime = class lifetime	Shared by all objects (one copy)
Local Variables	Inside methods/blocks	From declaration to end of block	Not visible outside the method

Type Casting:

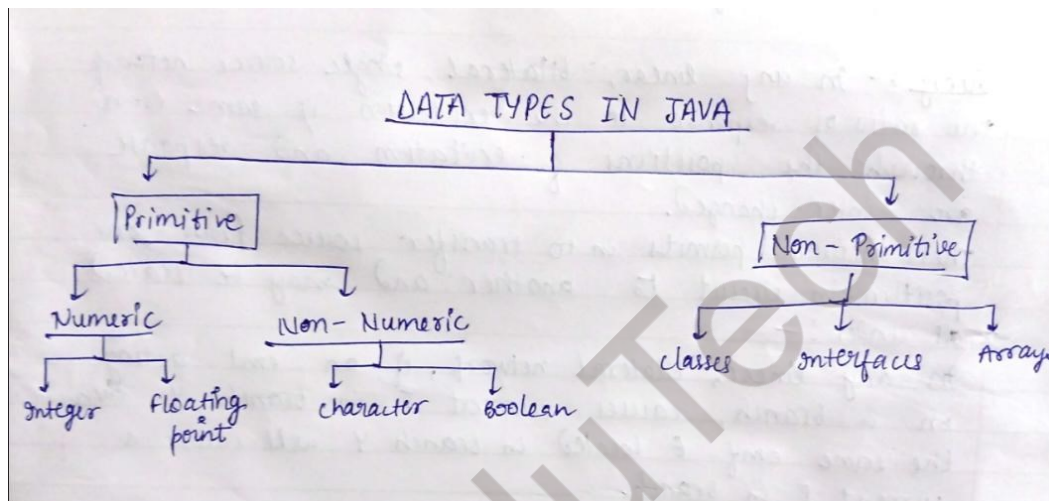
```
type var1 = (type) var2;
int m = 50;
byte n = (byte) m;    // Explicit casting
```

NOTE

Widening (small→large) is implicit. Narrowing (large→small) needs explicit cast using (type). Example: double→int needs (int) cast.

1.3 Data Types in Java

Java data types are divided into Primitive (built-in) – stores actual values, predefined, fixed size, faster, stored in stack and Non-Primitive (reference) – store reference(address) of objects, stored in heap.



Data Type	Category	Size	Range / Notes
byte	Primitive — Integer	1 byte (8 bits)	-128 to 127
short	Primitive — Integer	2 bytes (16 bits)	-32,768 to 32,767
int	Primitive — Integer	4 bytes (32 bits)	-2,147,483,648 to 2,147,483,647
long	Primitive — Integer	8 bytes (64 bits)	Very large integers — use 'L' suffix
float	Primitive — Float	4 bytes (32 bits)	~6-7 decimal digits — use 'f' suffix
double	Primitive — Float	8 bytes (64 bits)	~15-16 decimal digits (default decimal)
char	Primitive — Non-Numeric	2 bytes (16 bits)	Unicode characters: 0 to 65,535
boolean	Primitive — Non-Numeric	1 bit	true or false only
String	Non-Primitive (Class)	Variable	Sequence of characters — immutable
Arrays	Non-Primitive	Variable	Collection of same-type elements
Interface	Non-Primitive	Variable	Abstract type with method contracts

1.4 Wrapper Classes

Wrapper classes provide a way to use primitive data types as objects. Each primitive has a corresponding Wrapper class in java.lang package.

Primitive Type	Wrapper Class	Key Methods
byte	Byte	Byte.parseByte(str), byteValue()
short	Short	Short.parseShort(str), shortValue()
int	Integer	Integer.parseInt(str), intValue(), Integer.MAX_VALUE
long	Long	Long.parseLong(str), longValue()
float	Float	Float.parseFloat(str), floatValue()
double	Double	Double.parseDouble(str), doubleValue()
char	Character	Character.isDigit(c), isLetter(c), toUpperCase(c)
boolean	Boolean	Boolean.parseBoolean(str), booleanValue()

EXAM

Autoboxing = automatic conversion of primitive to wrapper. Unboxing = wrapper to primitive. Introduced in Java 5. Example: Integer i = 5; (autoboxing) | int x = i; (unboxing)

SECTION 2 — Arrays

An array is a group of like-typed variables that are referred to by a common name. Stored in continuous memory locations. Arrays in Java are objects. Steps to create an array: (1) Declare, (2) Create memory, (3) Initialize.

2.1 One-Dimensional Arrays

Declaration:

```
int number[] = new int[5]; // declare + create
```

Initialization:

```
number[0] = 35; number[1] = 40; number[2] = 20;
number[3] = 57; number[4] = 19;
```

Array Length:

```
int asize = a.length; // .length gives size of array
```

KEY

```
arrayname = new type[size]; // Create arrayname[subscript] = value; // Initialize
int asize = a.length; // Get length
```

2.2 Two-Dimensional Arrays

```
int myArray[][] = new int[3][4]; // 3 rows, 4 columns
myArray[0][0] = 10; myArray[1][2] = 25; // access element
```

2.3 Variable-Size (Jagged) Arrays

```
int x[][] = new int[3][];
x[0] = new int[2];    // row 0 has 2 columns
x[1] = new int[4];    // row 1 has 4 columns
x[2] = new int[3];    // row 2 has 3 columns
```

2.4 String Arrays

```
String itemArray[] = new String[3];
itemArray[0] = "Apple"; itemArray[1] = "Mango";
```

2.5 Important String Methods

Method	Description	Example
toLowerCase()	Converts string to lowercase	s1.toLowerCase()
toUpperCase()	Converts string to uppercase	s1.toUpperCase()
replace('x','y')	Replaces character x with y	s1.replace('a','b')
trim()	Removes leading & trailing spaces	s1.trim()
equals(s2)	Checks equality (case-sensitive)	s1.equals(s2)
equalsIgnoreCase(s2)	Checks equality ignoring case	s1.equalsIgnoreCase(s2)
length()	Returns length of string	s1.length()
charAt(n)	Returns char at index n	s1.charAt(0)
compareTo(s2)	Lexicographic comparison	s1.compareTo(s2)
concat(s2)	Concatenates s2 at end of s1	s1.concat(s2)
substring(n)	Substring from index n to end	s1.substring(3)
indexOf(s2)	First index of s2 in s1	s1.indexOf("hello")

SECTION 3 — Operators in Java

An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations.

3.1 Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 3	3 (integer), 3.33 (float)
%	Modulo (Remainder)	10 % 3	1

NOTE

Integer Arithmetic: Both operands integer → result integer. Real Arithmetic: Both operands real → result real. Mixed-Mode: One real, one integer → result real (integer promoted).

3.2 Relational Operators

Operator	Meaning	Example
<	Less than	$a < b$
<=	Less than or equal to	$a <= b$
>	Greater than	$a > b$
>=	Greater than or equal to	$a >= b$
==	Equal to	$a == b$
!=	Not equal to	$a != b$

3.3 Logical Operators

Operator	Name	Description
&&	Logical AND	True if BOTH operands are true. Short-circuits (stops at first false)
	Logical OR	True if AT LEAST ONE operand is true. Short-circuits (stops at first true)
!	Logical NOT	Reverses the boolean value. !true = false, !false = true

Truth Table:

op1	op2	op1 && op2	op1 op2
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

3.4 Assignment Operators

Operator	Equivalent	Example
=	Simple assignment	$a = 5$
+=	$a = a + \text{exp}$	$a += 1 \rightarrow a = a + 1$
-=	$a = a - \text{exp}$	$a -= 1 \rightarrow a = a - 1$
*=	$a = a * \text{exp}$	$a *= (n+1) \rightarrow a = a*(n+1)$
/=	$a = a / \text{exp}$	$a /= (n+1) \rightarrow a = a/(n+1)$
%=	$a = a \% \text{exp}$	$a \% = b \rightarrow a = a \% b$

3.5 Increment & Decrement Operators

Type	Operator	Meaning	Example (m=5)	Result
Pre-increment	++m	Increment BEFORE use	y = ++m;	y=6, m=6
Post-increment	m++	Increment AFTER use	y = m++;	y=5, m=6
Pre-decrement	--m	Decrement BEFORE use	y = --m;	y=4, m=4
Post-decrement	m--	Decrement AFTER use	y = m--;	y=5, m=4

3.6 Conditional (Ternary) Operator

Syntax: expression1 ? expression2 : expression3

If expression1 is true → result is expression2, else result is expression3.

```
a = 10; b = 15;
x = (a > b) ? a : b; // x = 15 (b is larger)
```

3.7 Bitwise Operators

Operator	Name	Description
&	Bitwise AND	1 if both bits are 1
	Bitwise OR	1 if at least one bit is 1
^	Bitwise XOR (exclusive OR)	1 if bits are different
~	One's Complement (NOT)	Inverts all bits
<<	Left Shift	Shifts bits left, fills with 0s (multiplies by 2)
>>	Right Shift	Shifts bits right, fills with sign bit (divides by 2)
>>>	Unsigned Right Shift	Shifts right, fills with 0 (ignores sign)

3.8 Special Operators

instanceof Operator:

Returns true if the object on the left side is an instance of the class on the right side.

```
person instanceof Student // true if person belongs to Student class
```

Dot Operator (.):

Used to access instance variables and methods of class objects.

```
person1.age // access field
person1.salary() // access method
```

BODMAS Rule for Arithmetic Expressions:

Brackets → Of → Division → Multiplication → Addition → Subtraction

```
x = a * b - c;
y = b / c * a;
z = a - b / c + d; // b/c is done first due to precedence
```

SECTION 4 — Control of Flow

Control statements determine the order in which statements are executed. Types: Decision-making (if, switch), Looping (while, do-while, for), Jump (break, continue, return).

4.1 Decision Making — if Statement

Type	Syntax / Structure
Simple if	if (test expression) { statement-x; } <pre> int age = 18; if(age >= 18){ System.out.println("You can vote"); } </pre>
if-else	if (test expression) { ... } else { ... } <pre> int age = 16; if(age >= 18){ System.out.println("You can vote"); } else { System.out.println("You cannot vote"); } </pre>
Nested if-else	if (cond) { if (cond) {} else {} } else {} <pre> int num = 10; if(num > 0){ if(num % 2 == 0){ System.out.println("Positive Even Number"); } else { System.out.println("Positive Odd Number"); } } else { System.out.println("Negative Number"); } </pre>

Type	Syntax / Structure
else-if ladder	<pre> if(c1){ } else if(c2){ } else if(c3){ } else { } int marks = 85; if(marks >= 90){ System.out.println("A grade"); } else if(marks >= 75){ System.out.println("B grade"); } else { System.out.println("C grade"); } </pre>

else-if Ladder Example:

```

if (condition1) { statement-1; }
else if (condition2) { ... }
else if (condition3) { ... }
else { statement-x; }

```

4.2 Switch Statement

Used for multi-way branching. The expression is matched against case values. break exits the switch. default handles unmatched cases.

```

switch (expression) {
    case value-1: block-1; break;
    case value-2: block-2; break;
    default:      default-block; break;
}

int day = 2;

switch(day){
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Invalid");
}

```

NOTE

Without break, execution falls through to the next case. Switch expression can be: int, char, String (Java 7+), or enum.

4.3 Looping Statements

Loop	Syntax	When to Use	Check
while	while (condition) { body; }	When number of iterations unknown	Pre-check (may not execute at all)
do-while	do { body; } while (condition);	When body must execute at least once	Post-check (always executes once)
for	for (init; condition; increment) { }	When number of iterations known	Pre-check

while Loop:

```
Initialization;
while (test condition) {
    body of the loop;
}
```

```
int i = 1;

while(i <= 5){
    System.out.println(i);
    i++;
}
```

do-while Loop:

```
Initialization;
do {
    body;
} while (test condition);
```

```
int i = 1;

do{
    System.out.println(i);
    i++;
} while(i <= 5);
```

for Loop:

```
for (initialization; test condition; increment) {
    body;
}
```

```
for(int i = 1; i <= 5; i++){
    System.out.println(i);
}
```

EXAM

Key difference: while and for check condition BEFORE execution (pre-test). do-while checks AFTER (post-test), so it always executes at least once.

4.4 Jump Statements

break

```
for(int i = 1; i <= 5; i++){
    if(i == 3){
        break;
    }
    System.out.println(i);
}
```

continue

```
for(int i = 1; i <= 5; i++){
    if(i == 3){
        continue;
    }
    System.out.println(i);
}
```

return

```
public static int add(int a, int b){
    return a + b;
}
```

SECTION 5 — Classes, Objects, and Methods

5.1 Defining a Class

```
class ClassName [extends SuperclassName] {
    [fields declaration;]
    [methods declaration;]
}
```

Eg:-

```
class Student {
    String name;
    int age;
}
```

Example — Rectangle Class:

```
class Rectangle {
    int length, width;           // field declarations
    void getData(int x, int y) { // method
        length = x; width = y;
    }
}
```

5.2 Creating Objects

Eg:-

```
Student s1 = new Student();  
s1.name = "ABC";  
s1.age = 24;
```

```
Rectangle rect1;           // Step 1: Declare object  
rect1 = new Rectangle();   // Step 2: Instantiate  
// OR in one line:  
Rectangle rect1 = new Rectangle();
```

Accessing Class Members:

```
rect1.length = 15;         // access field  
rect1.width = 10;  
rect1.getData(15, 10);     // call method
```

5.3 Methods

```
class Student {  
    String name;  
  
    void display() {  
        System.out.println("Name: " + name);  
    }  
}
```

Use of Method:

```
Student s1 = new Student();  
s1.name = "Namrata";  
s1.display();
```

5.4 Constructors

Constructors have the same name as the class. They do NOT specify a return type (not even void). They are called automatically when an object is created with 'new'. They initialize the object's instance variables.

```
class Student {  
    String name;  
  
    Student() { // constructor  
        name = "Default Name";  
    }  
}
```

```
class Rectangle {
    int length, width;
    Rectangle(int x, int y) {    // constructor
        length = x;  width = y;
    }
    int rectArea() { return length * width; }
}
// Usage:
Rectangle r = new Rectangle(15, 10);
int area = r.rectArea();    // area = 150
```

5.5 Method Overloading

In Java, multiple methods can have the same name but different parameter lists (different number or types of parameters). This is called Method Overloading. Used when objects need to perform similar tasks with different inputs.

```
class MathOp {
    int add(int a, int b){
        return a + b;
    }

    int add(int a, int b, int c){
        return a + b + c;
    }
}
```

Example — Room Class with Overloaded Constructors:

```
class Room {
    float length, breadth;
    Room(float x, float y) { length=x; breadth=y; }    // 2 params
    Room(float x)      { length=breadth=x; }           // 1 param (square)
    int area()          { return (int)(length*breadth); }
}
```

5.6 Static Members

Static variables and methods belong to the CLASS itself rather than to individual objects. Also called class variables and class methods.

```
class Student {
    static String college = "IPU";
}
```

Feature	Static (Class) Member	Instance Member
Belongs to	Class itself	Individual object
Memory	One copy shared by all objects	Separate copy for each object
Access	ClassName.member	objectName.member
Call without object?	Yes — ClassName.method()	No — needs object
Keyword	static modifier	No special keyword

5.7 Class Member Modifiers

Modifier	Applied To	Effect
public	class, field, method	Accessible from everywhere
private	field, method	Accessible only within the same class
protected	field, method	Accessible in same package + subclasses in other packages
(default)	field, method, class	Accessible within same package only (no keyword needed)
static	field, method	Belongs to class, shared by all objects, accessible without object
final	variable, method, class	Variable: constant. Method: cannot override. Class: cannot subclass
abstract	method, class	Method has no body (must override). Class cannot be instantiated

Visibility Control Table (from your notes):

Access From	public	protected	default	private-protected	private
Same class	Yes	Yes	Yes	Yes	Yes
Subclass (same package)	Yes	Yes	Yes	Yes	No
Other class (same package)	Yes	Yes	Yes	No	No
Subclass (other package)	Yes	Yes	No	Yes	No
Non-subclass (other package)	Yes	No	No	No	No

5.8 Final Variables and Classes

- final variable = constant — value cannot be changed after initialization

```
final int SIZE = 100; // SIZE is a constant
```

- final method = cannot be overridden in subclasses
- final class = cannot be subclassed (e.g., String class is final)

```
final class AClass { ... } // no class can extend AClass
```

5.9

Finalize Method

Java supports finalization which is opposite to initialization. The finalize() method is called by the Garbage Collector before an object is removed from memory to free up resources like file descriptors or window system fonts.

```
protected void finalize() {
```

```
// cleanup code here
}
```

NOTE

finalize() is deprecated in Java 9 and removed in Java 18. Use try-with-resources instead for resource management.

SECTION 6 — Inheritance

The mechanism of deriving a new class from an old one is called Inheritance. The old class is known as the base class / super class / parent class, and the new one is called subclass / derived class / child class.

6.1 Types of Inheritance

Type	Description	Java Support	Keyword
Single	One subclass from one superclass (A → B)	Yes	extends
Multiple	One subclass from multiple superclasses (A,B → C)	No (via Interface only)	implements
Hierarchical	One superclass, many subclasses (A → B, C, D)	Yes	extends
Multilevel	Derived from a derived class (A → B → C)	Yes	extends
EXAM	Java does NOT support multiple inheritance through classes to avoid the Diamond Problem. It is achieved through Interfaces. Remember: classes use 'extends', interfaces use 'implements'.		

6.2 Defining a Subclass

```
class SubclassName extends SuperclassName {
    // additional variables
    // additional methods
}
```

6.3 Subclass Constructor & super Keyword

A subclass constructor uses the keyword super to invoke the constructor method of the superclass.

- super() must be used within a subclass constructor method
- **The call to superclass constructor must appear as the FIRST statement**
- Parameters in super() must match the order and type of instance variables in superclass

```
class Super { int x;
    Super(int x) { this.x = x; }
    void display() { System.out.println("Super x="+x); } }
class Sub extends Super { int y;
    Sub(int x, int y) { super(x); this.y = y; } // super() first
    void display() {
        System.out.println("Super x="+x);
        System.out.println("Sub y="+y); } }
// Output: Super x=100    Sub y=200
```

6.4 Method Overriding

When a subclass defines a method with the SAME name, SAME parameters, and SAME return type as a method in the superclass, the subclass method overrides the superclass method.

Feature	Overloading	Overriding
Same method name	Yes	Yes
Parameters	Must be DIFFERENT	Must be SAME
Return type	Can differ	Must be SAME
Where	Same class	Subclass
Resolved at	Compile time	Runtime
Also called	Compile-time polymorphism	Runtime polymorphism

6.5 Dynamic Method Dispatch (Runtime Polymorphism)

Dynamic Method Dispatch is the mechanism in Java to implement Runtime Polymorphism. When an overridden method is called through a superclass reference variable, Java determines which version of the method to call at RUNTIME based on the actual object type.

```
Super A = new Sub();           // Superclass ref, Sub object
A.method();                     // Sub's version called at runtime!
// Output: Method Sub
```

EXAM

Key rule: The TYPE of the OBJECT (not the reference variable) determines which overridden method gets called. This is resolved at runtime — hence 'dynamic'.

6.6 Abstract Methods & Abstract Classes

A class declared with the abstract keyword is an abstract class. It can have abstract methods (no body) and non-abstract methods.

- **Abstract class CANNOT be instantiated (cannot use new)**
- It must be extended and its abstract methods implemented by subclass
- If subclass does not implement all abstract methods, it must also be abstract

```
abstract class Shape {
    abstract void draw();           // abstract method - no body
    void color() {                  // concrete method - has body
        System.out.println("Colored"); }
}
class Circle extends Shape {
    void draw() { System.out.println("Drawing circle"); }
}
```

SECTION 7 — Interfaces and Anonymous Inner Classes

7.1 What is an Interface?

An interface is basically a kind of class. Like classes, interfaces contain methods and variables but with a major difference — interfaces define **ONLY** abstract methods and **FINAL** fields. They do not contain any implementation code. A class that implements an interface must define the code for all methods declared in the interface.

Feature	Abstract Class	Interface
Methods	Both abstract & concrete	Only abstract (Java 7), default/static (Java 8+)
Variables	Any type	Only public static final (constants)
Constructor	Yes	No
Inheritance	Single (extends)	Multiple (implements many)
Instantiation	No	No
Access modifiers	Any	public only

7.2 Defining an Interface

```
interface InterfaceName {
    variables declaration;    // public static final (implicit)
    methods declaration;      // public abstract (implicit)
}
```

Example:

```
interface Area {
    final static float pi = 3.14f; // constant
    float compute(float x, float y); // abstract method
}
```

7.3 Implementing an Interface

```
class ClassName extends SuperClass implements Interface1, Interface2 {
    // must implement ALL interface methods
}
```

Complete Example:

```
class Rectangle implements Area {
    public float compute(float x, float y) { return x*y; }
}
class Circle implements Area {
    public float compute(float x, float y) { return pi*x*x; }
}
// Output: Area of rectangle=200    Area of circle=314
```

7.4 Extending Interfaces

Interfaces can extend other interfaces using the **extends** keyword. A class implementing a child interface must implement methods from both parent and child interfaces.

```
interface ItemConstants { int code=1001; String name="Fan"; }
interface ItemMethods { void display(); }
interface Item extends ItemConstants, ItemMethods { ... }
```

7.5 Anonymous Inner Classes

An anonymous inner class is a local class without a name. It is defined and instantiated in a single expression. Used when you need a one-time-use implementation of an interface or abstract class.

```
// Anonymous class implementing an interface:
Runnable r = new Runnable() {
    public void run() {
        System.out.println("Running!");
    }
};

// Anonymous class extending an abstract class:
Shape s = new Shape() {
    void draw() { System.out.println("Drawing"); }
};
```

KEY

Anonymous inner class = class defined inside an expression. No name, no separate class file visible to programmer. Commonly used in GUI event handling.

SECTION 8 — Exception Handling

An exception is a condition that is caused by a run-time error in the program. When the Java interpreter encounters an error (like dividing by zero), it creates an exception object and throws it.

Four Steps of Exception Handling:

- Find the problem (Hit the exception)
- Inform that an error has occurred (Throw the exception)
- Receive the error information (Catch the exception)
- Take corrective actions (Handle the exception)

8.1 Types of Errors

Error Type	Detected By	Examples	Fix
Compile-time errors	Java Compiler	Missing semicolons, brackets, misspelled keywords, incompatible types, using = instead of ==	Fix before running — no .class file created
Run-time errors	JVM at runtime	Divide by zero, array out of bounds, null pointer, stack overflow, invalid cast	Handle with exceptions

8.2 Checked vs Unchecked Exceptions

Type	Description	Examples	Extends
Checked	Explicitly handled in code using try-catch	IOException, FileNotFoundException, SQLException	java.lang.Exception
Unchecked	Not required to handle; JVM handles if not caught	ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException	java.lang.RuntimeException

8.3 Common Java Exceptions

Exception	Caused By
ArithmeticException	Division by zero (int/int)
ArrayIndexOutOfBoundsException	Accessing invalid array index
ArrayStoreException	Storing incompatible type in array
FileNotFoundException	File not found during file operations
IOException	Input/output operation failure
NullPointerException	Using a null object reference
OutOfMemoryException	JVM heap exhausted
StackOverflowError	Infinite recursion — stack full
ClassCastException	Illegal type cast
NumberFormatException	Invalid string-to-number conversion

8.4 try-catch-finally Syntax

```
try {
    // code that may generate exception
}
catch (ExceptionType e) {
    // code to process the exception
}
finally {
    // always executes (cleanup code)
}
```

Multiple catch blocks:

```
try { ... }
catch (ExceptionType1 e) { ... }
catch (ExceptionType2 e) { ... }
catch (ExceptionType-n e) { ... }
```

EXAM

finally block ALWAYS executes — even if an exception occurs or a return statement is inside try. Used for cleanup: closing files, DB connections etc.

Complete Example:

```
class Errors3 {
    public static void main(String args[]) {
        int a=10, b=5, c=5;
        try {
            int x = a/(b-c); // division by zero!
        }
        catch (ArithmeticException e) {
            System.out.println("Division by zero");
        }
        int y = a/(b+c);
        System.out.println("y = " + y); // y = 1
    }
}
```

}

8.5 throw and throws Keywords

Keyword	Purpose	Syntax	Used Where
throw	Explicitly THROW an exception object	throw new ExceptionType(msg);	Inside method body
throws	DECLARE that a method may throw an exception	type method() throws ExType { }	In method signature

throws Example:

```
static void divide_m() throws ArithmeticException {
    int x=22, y=0, z;
    z = x/y;    // will throw ArithmeticException
}
public static void main(String args[]) {
    try { divide_m(); }
    catch (ArithmeticException e) {
        System.out.println("Caught the exception: " + e); }
}
```

8.6 User-Defined Exceptions

We can create our own exception classes by extending the Exception class (for checked) or RuntimeException (for unchecked).

```
import java.lang.Exception;
class MyException extends Exception {
    MyException(String message) {
        super(message);    // pass message to parent Exception
    }
}
class TestMyException {
    public static void main(String args[]) {
        int x=5, y=1000;
        try {
            float z = (float)x / (float)y;
            if (z < 0.01) {
                throw new MyException("No. is too small");
            }
        }
        catch (MyException e) {
            System.out.println("Caught exception");
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("I am always here");
        }
    }
}
```

SECTION 9 — StringBuffer Class and StringTokenizer

9.1 StringBuffer Class

StringBuffer is a peer class of String. While String creates strings of FIXED length (immutable), StringBuffer creates strings of FLEXIBLE length that can be modified in terms of both length and content. We can insert characters & substrings in the middle or append to the end.

Feature	String	StringBuffer
Mutability	Immutable (cannot change)	Mutable (can change)
Thread Safety	Thread-safe	Thread-safe (synchronized)
Performance	Slow for modifications (creates new object)	Fast for modifications (modifies in-place)
Storage	String constant pool	Heap

StringBuffer Methods:

Method	Description	Example
setCharAt(n, 'x')	Modifies the nth character to 'x'	sb.setCharAt(0,'J')
append(s2)	Appends string s2 at the END of sb	sb.append(" World")
insert(n, s2)	Inserts string s2 at position n of sb	sb.insert(5,"Java")
setLength(n)	Sets length. If n < current length: truncates. If n > current: adds nulls	sb.setLength(10)
delete(start,end)	Deletes chars from start to end-1	sb.delete(2,5)
reverse()	Reverses the string	sb.reverse()
toString()	Converts StringBuffer to String	sb.toString()
length()	Returns current length	sb.length()
charAt(n)	Returns char at index n	sb.charAt(3)

9.2 StringTokenizer

StringTokenizer class (in java.util) is used to break a string into tokens (words/parts) based on delimiters. Default delimiter is whitespace (space, tab, newline).

```
import java.util.StringTokenizer;
StringTokenizer st = new StringTokenizer("Hello World Java");
while (st.hasMoreTokens()) {
    System.out.println(st.nextToken());
}
// Output: Hello    World    Java
```

Method	Description
hasMoreTokens()	Returns true if more tokens are available
nextToken()	Returns next token (as String)
countTokens()	Returns number of remaining tokens



Method	Description
nextToken(delim)	Returns next token using specified delimiter
NOTE	StringTokenizer is a legacy class. For modern Java, use String.split() method instead: String[] parts = str.split(" ");

SECTION 10 — Packages

Packages are Java's way of grouping a variety of classes and/or interfaces together. The grouping is done according to functionality. Packages act as 'containers' for classes.

Type	Description	Examples
Java API Packages	Built-in packages provided by Java	java.lang, java.util, java.io, java.awt, java.net, java.applet
User-defined Packages	Created by programmer for organization	package mypackage;

Creating a Package:

```
package firstPackage;           // first line of source file
public class FirstClass { ... }
```

Accessing / Using Packages:

```
import packagename.*;           // import all classes
import package1.ClassA;         // import specific class
// Full path: double y = java.lang.Math.sqrt(x);
```

SECTION 11 — Applets, Life Cycle & Security Concerns

11.1 What are Applets?

Applets are small Java programs that are primarily used in Internet computing. An applet can perform arithmetic operations, display graphics, play sounds, accept user input, create animation, and play interactive games.

Type	Description
Local Applet	Developed locally, stored on local system. Web page finds it without using the Internet.
Remote Applet	Developed by someone else, stored on remote computer connected to Internet. Downloaded via Internet and run locally.

Applet class hierarchy:

```
java.lang.Object → java.awt.Component → java.awt.Container
→ java.awt.Panel → java.applet.Applet
```

11.2 Steps to Develop an Applet

- Step 1: Build an applet code (.java file)
- Step 2: Create an executable applet (.class file) using javac
- Step 3: Design a web page using HTML tags
- Step 4: Prepare the <APPLET> tag
- Step 5: Incorporate <APPLET> tag into the webpage
- Step 6: Create the HTML file
- Step 7: Test the applet code using appletviewer or browser

11.3 Hello Java Applet — Complete Example

```
import java.awt.*;
import java.applet.*;
public class HelloJava extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello Java", 10, 100);
    }
}
// Save as: HelloJava.java
// Compile: javac HelloJava.java
```

HTML File:

```
<HTML><HEAD><TITLE>Welcome to Java Applets</TITLE></HEAD>
<BODY><CENTER>
<APPLET CODE="HelloJava.class" HEIGHT=400 WIDTH=400>
</APPLET></CENTER></BODY></HTML>
// Save as: HelloJava.html
// Run: appletviewer HelloJava.html
```

11.4 Applet Life Cycle — 5 States (MOST IMPORTANT!)

An applet goes through 4 states in its lifetime: Born → Running → Idle → Dead. The display state is part of Running.

State	Method Called	Triggered When	Description
Born / Initialization	init()	Applet first loaded	Called ONCE. Initializes variables, creates UI components. Like constructor.
Running	start()	After init(); also when user returns to page	Called to start or resume execution. May be called multiple times.
Display	paint()	When applet needs to draw/redraw	Performs output on screen. Called whenever display needs updating.
Idle / Stopped	stop()	User leaves the page	Suspends execution. Can restart with start(). Called multiple times.
Dead	destroy()	Browser closes / applet removed	Called ONCE. Final cleanup. Object removed from memory.

Life Cycle Methods:

```
public void init()    { /* one-time initialization */ }
public void start()   { /* start or resume execution */ }
public void paint(Graphics g) { /* draw output */ }
public void stop()    { /* suspend execution */ }
public void destroy() { /* final cleanup */ }
```

EXAM

Life Cycle Order: init() → start() → paint() → stop() → destroy(). init() and destroy() called ONCE each. start()/stop()/paint() can be called multiple times. This is asked in EVERY exam!

11.5 Passing Parameters to Applets

```
// In Applet Java code:
str = getParameter("string"); // read HTML param
if (str == null) str = "Java"; // default value
```

```
// In HTML file:
<APPLET CODE="HelloJavaParam.class" WIDTH=400 HEIGHT=200>
  <PARAM NAME="string" VALUE="Applet!">
</APPLET>
```

11.6 Applet Security Concerns

Applets (especially remote ones) pose security risks because they run on the local machine with access to local resources. Java implements a Sandbox Model to restrict what applets can do.

What Untrusted Applets CANNOT Do:

- Read or write to the local file system
- Make network connections to any host except the originating server
- Load native libraries or execute local programs
- Access system properties (like user.home, user.name)
- Create new class loaders
- Define system classes or read system clipboard

Security Mechanisms:

Mechanism	How It Helps
Bytecode Verifier	Checks downloaded .class files for validity and type safety before execution
Class Loader	Separates local trusted code from remote untrusted code; prevents spoofing
Security Manager	Intercepts and controls all sensitive runtime operations; enforces sandbox rules
Digital Signatures	Signed applets from trusted sources can be granted additional permissions
Security Policy File	Defines specific permissions for specific code sources

NOTE

Java Applets are deprecated since Java 9 and completely removed in Java 17. Modern web uses JavaScript/HTML5. However, for IPU exam purposes, know the complete applet lifecycle and security model.

SECTION 12 — Most Important PYQs and Exam Tips

Q1

Explain the life cycle of an applet with diagram. (5-10 marks) [REPEATED EVERY YEAR]

Draw state diagram: Born→Running(paint)→Idle→Dead. Explain all 5 methods: init(), start(), paint(), stop(), destroy(). Mention init() and destroy() called once only.

Q2

What is the difference between interface and abstract class? (5 marks)

Table: Methods (only abstract vs both), Variables (final static vs any), Constructor (No vs Yes), Multiple inheritance (Yes vs No). Give syntax of each.

Q3

Explain exception handling in Java with try-catch-finally. (5-10 marks)

Define exception. Checked vs unchecked. Syntax of try-catch-finally. Example of ArithmeticException. Multiple catch blocks. finally always executes.

Q4

Write a Java program to demonstrate method overriding / dynamic method dispatch. (5 marks)

Write Super class with method(), Sub class overriding method(). In main: Super ref = new Sub(); ref.method(); Output: Sub's method is called at runtime.

Q5

Explain the types of inheritance in Java with diagrams. (5 marks)

Single, Multilevel, Hierarchical, Multiple (via interface). Draw box diagrams for each. Note: Java doesn't support multiple inheritance via classes.

Q6

What is an interface? How is it different from a class? Write a program to implement an interface. (10 marks)

Define interface. Differences table. Area interface example with Rectangle and Circle implementing it. Show compute() method implementation.

Q7

Explain user-defined exceptions in Java with an example. (5 marks)

Define user exception. Steps: extend Exception class, write constructor with super(message). throw new MyException(msg). catch(MyException e){ e.getMessage(); }

Q8**Explain the StringBuffer class and its methods. (5 marks)**

String vs StringBuffer (mutable/immutable). Methods: append(), insert(), delete(), reverse(), setCharAt(), setLength(), toString(). Give examples.

Q9**Differentiate between overloading and overriding. (3-5 marks)**

Table: Parameters (different vs same), Return type (can differ vs same), Where (same class vs subclass), Resolved (compile time vs runtime), Type (static vs dynamic polymorphism).

Q10**Write a Java program to demonstrate exception handling using throw and throws. (5 marks)**

Write method with 'throws ArithmeticException'. Inside method, throw ArithmeticException. In main, call method inside try-catch. Show output.

Q11**What are wrapper classes? Explain with examples. (5 marks)**

Table of 8 primitives and their wrappers. Purpose: use primitive as object, collection framework. Autoboxing/unboxing. Integer.parseInt(), Double.parseDouble() examples.

Q12**Explain the applet security concerns. (5 marks)**

Sandbox model. What applets cannot do (file access, network, native libraries). Three security mechanisms: Bytecode Verifier, Class Loader, Security Manager.

All the Best for Your IPU End-Sem Exams! Go Ace It!

Revise these notes 2-3 times • Draw diagrams • Write code programs by hand